



Karol Mazur

Kierunek: informatyka

Specjalizacja: sztuczna inteligencja

Numer albumu: 394248

Algorytmy kwantowe dla wybranych problemów grafowych

Praca magisterska

wykonana pod kierunkiem

dr hab. Kordiana A. Smolińskiego

w Katedrze Fizyki Teoretycznej

WFiIS UŁ

Łódź 2026

Spis treści

1	Wstęp	3
2	Podstawy teoretyczne	4
2.1	Problem najkrótszej ścieżki	4
2.2	Algorytm Dijkstry	4
2.3	Znajdowanie minimum	6
2.3.1	Wersja naiwna	6
2.3.2	Wersja z kopcem binarnym	7
2.3.3	Wersja kwantowa	8
2.4	Rodzaje grafów	11
2.4.1	Graf gęsty (Dense Graph)	12
2.4.2	Graf o średniej gęstości (Half Dense)	13
2.4.3	Graf rzadki (Sparse Graph)	14
2.4.4	Przypadek szczególny (Special Case)	15
2.5	Reprezentacja grafów	16

Wstęp

Duża klasa problemów życia codziennego może być modelowanych przez odpowiadające im problemy grafowe. Wiele problemów grafowych jest NP-zupełnych lub NP-trudnych; są też problemy, dla których istnieją algorytmy wielomianowe. Dla wielu tych algorytmów kluczowym etapem jest przeprowadzenie wyszukiwania, np. minimalnej drogi. Ten etap można znacząco przyspieszyć wykorzystując kwantowy algorytm wyszukiwania minimum, będący uogólnieniem znanego kwantowego algorytmu wyszukiwania Grovera.

Celem tej pracy jest porównanie klasycznego i kwantowego wariantu algorytmu Dijkstry, służącego do znajdowania najkrótszej drogi w grafie. Algorytm ten zostanie zaimplementowany na trzy różne sposoby, a następnie zostanie sprawdzone, jak te podejścia radzą sobie na różnych rodzajach i rozmiarach grafów.

W pierwszej części pracy przedstawione zostaną podstawowe zagadnienia teoretyczne, obejmujące definicję grafów, ich rodzaje wykorzystywane w przeprowadzonych badaniach oraz zasadę działania algorytmu Dijkstry. Omówione zostaną również różne sposoby implementacji tego algorytmu. W dalszej części zaprezentowane zostaną wyniki przeprowadzonych eksperymentów wraz z ich analizą statystyczną, umożliwiającą ocenę rzeczywistej wydajności poszczególnych implementacji w zależności od charakterystyki analizowanych grafów. Na końcu pracy zamieszczono instrukcję instalacji i konfiguracji środowiska oraz wymaganych bibliotek, umożliwiającą samodzielne uruchomienie aplikacji i odtworzenie przedstawionych eksperymentów.

Podstawy teoretyczne

2.1 Problem najkrótszej ścieżki

Problem wyznaczania najkrótszej ścieżki należy do podstawowych zagadnień teorii grafów. Jego celem jest znalezienie ścieżki pomiędzy dwoma wierzchołkami grafu, której suma wag wszystkich krawędzi jest minimalna. Problem ten znajduje zastosowanie w wielu dziedzinach informatyki, takich jak systemy nawigacji, routing w sieciach komputerowych, logistyka, planowanie tras czy analiza sieci społecznościowych. Formalnie graf definiowany jest jako para

$$G = (V, E) \quad (2.1)$$

gdzie (V) oznacza zbiór wierzchołków, natomiast (E) zbiór krawędzi łączących pary wierzchołków. W przypadku grafów ważonych każdej krawędzi przypisana jest waga określająca koszt przejścia pomiędzy dwoma sąsiadującymi wierzchołkami. Jeżeli wszystkie wagi są nieujemne, jednym z najbardziej efektywnych algorytmów rozwiązujących problem najkrótszych ścieżek z początkowego wierzchołka jest algorytm Dijkstry.

2.2 Algorytm Dijkstry

Algorytm Dijkstry [3] został po raz pierwszy przedstawiony w 1959 roku przez Edsger W. Dijkstra. Jest to algorytm zachłanny, który znajduje najkrótsze ścieżki od jednego wierzchołka źródłowego do wszystkich pozostałych wierzchołków w grafie o nieujemnych wagach krawędzi. Podstawową ideą algorytmu jest stopniowe powiększanie zbioru odwiedzonych wierzchołków, dla których wyznaczono już najkrótszą możliwą odległość od źródła. W każdej kolejnej iteracji wybierany jest wierzchołek o najmniejszej znanej odległości spośród nieodwiedzonych, po czym wykonywana jest operacja relaksacji wszystkich wychodzących z niego krawędzi.

Relaksacja polega na sprawdzeniu, czy przejście przez aktualnie analizowany wierzchołek prowadzi do krótszej ścieżki do sąsiada. Jeżeli zachodzi zależność

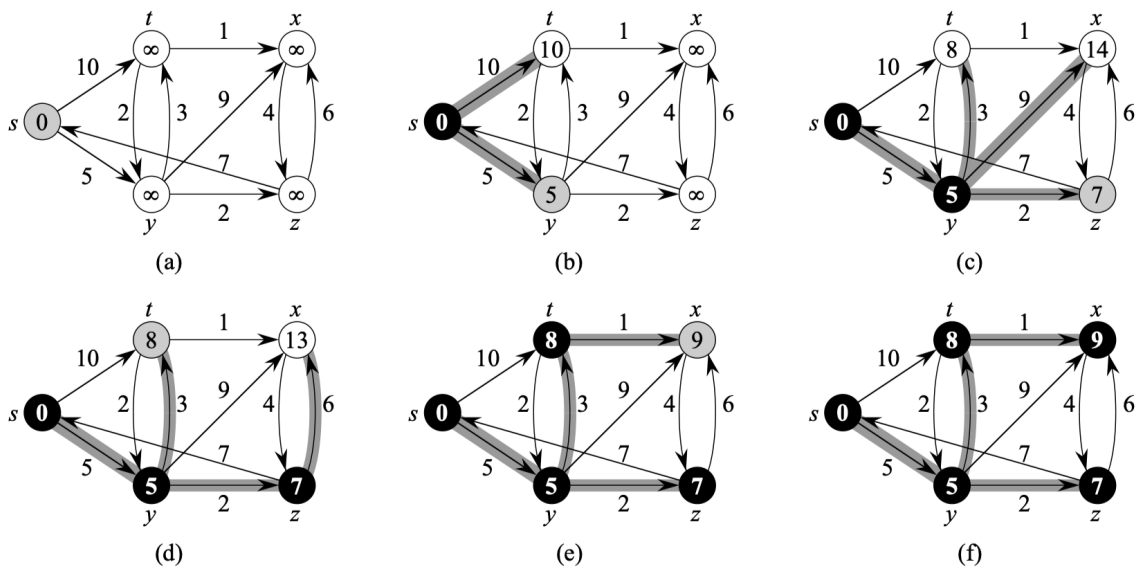
$$d(u) + w(u, v) < d(v) \quad (2.2)$$

wartość ($d(v)$) zostaje zaktualizowana. Algorytm kończy działanie po odwiedzeniu wszystkich osiągalnych wierzchołków.

ALGORYTM DIJKSTRY

1. Zainicjalizuj odległości wszystkich wierzchołków od wierzchołka źródłowego.
2. Utwórz pusty zbiór odwiedzonych wierzchołków $S \leftarrow \emptyset$.
3. Umieść wszystkie wierzchołki grafu w zbiorze Q .
4. Dopóki zbiór Q nie jest pusty, wykonuj:
 - (a) Wybierz i usuń ze zbioru Q wierzchołek u o najmniejszej aktualnej odległości od źródła.
 - (b) Dodaj wierzchołek u do zbioru odwiedzonych S .
 - (c) Dla każdego sąsiada v wierzchołka u wykonaj relaksację krawędzi (u, v) .

Opracowanie własne na podstawie [2, Chapter 24.3].



Rysunek 2.1: Przykład działania algorytmu Dijkstry. [2, Rozdział 24.3]

2.3 Znajdowanie minimum

Najbardziej kosztownym etapem algorytmu Dijkstry jest wyznaczenie kolejnego nieodwiedzonego wierzchołka o najmniejszej odległości. Sposób w jaki jest to zaimplementowane znacząco wpływa na złożoność całego algorytmu Dijkstry. Dlatego w celu porównania różnych podejść w niniejszej pracy wykorzystano trzy wersje algorytmu Dijkstry z różnymi sposobami znajdowania minimum:

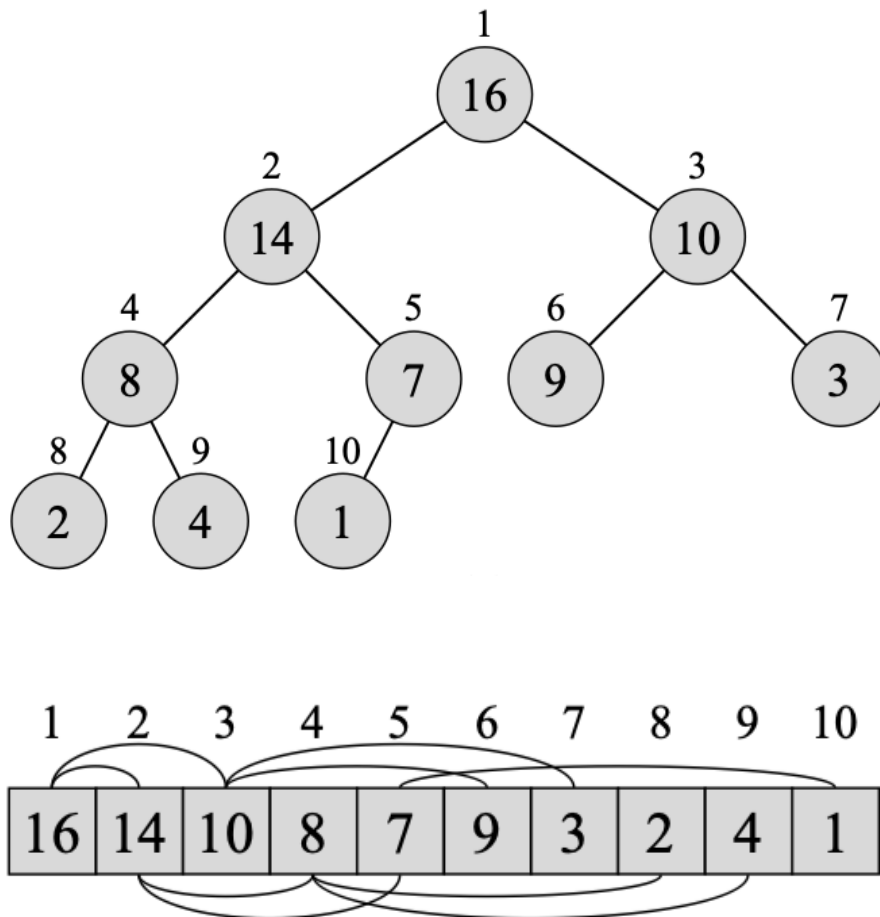
- wersję naiwną,
- wersję z wykorzystaniem kopca binarnego,
- wersję kwantową opartą na algorytmie wyszukiwania minimum Dürra i Høyer [4].

2.3.1 Wersja naiwna

W wersji naiwnej wybór kolejnego wierzchołka realizowany jest poprzez liniowe przeszukanie tablicy odległości w celu znalezienia najmniejszej wartości spośród wszystkich nieodwiedzonych wierzchołków. Operacja ta jest wykonywana w każdej iteracji algorytmu Dijkstry, a ponieważ liczba iteracji jest proporcjonalna do liczby wierzchołków V , całkowity koszt wyszukiwania minimum wynosi $O(V^2)$. Pomimo niskiej wydajności, wersja naiwna cechuje się prostotą implementacji i często wykorzystywana jest jako punkt odniesienia podczas porównywania bardziej zaawansowanych wariantów algorytmu jak na przykład kopiec binarny.

2.3.2 Wersja z kopcem binarnym

W celu przyspieszenia operacji wyboru kolejnego wierzchołka można zastosować kopiec binarny przechowujący pary składające się z identyfikatora wierzchołka oraz aktualnej wartości odległości. Kopiec binarny jest strukturą danych reprezentowaną w postaci niemal pełnego drzewa binarnego, w którym każdy węzeł posiada co najwyżej dwoje dzieci. W kopcu minimalnym spełniona jest własność, zgodnie z którą wartość klucza każdego węzła jest nie większa od wartości kluczy jego potomków. Oznacza to, że element o najmniejszym kluczu znajduje się zawsze w korzeniu drzewa. Jego pobranie oraz usunięcie wymaga czasu $O(\log V)$. Wstawienie nowego elementu lub aktualizacja wartości klucza również wykonywane są w czasie $O(\log V)$ poprzez odpowiednie odtworzenie własności kopca. Dzięki zastosowaniu kopca binarnego wyszukiwanie wierzchołka o najmniejszej odległości nie wymaga liniowego przeszukiwania wszystkich elementów.[2, Rozdział 6]



Rysunek 2.2: Przykład kopca binarnego max wraz z listą jego wartości i kluczy. [2, Rozdział 6] W niniejszej pracy do implementacji Dijkstry jest wykorzystywana wersja min kopca binarnego

2.3.3 Wersja kwantowa

W implementacji kwantowej klasyczna operacja wyboru nieodwiedzonego wierzchołka o najmniejszej wartości odległości została zastąpiona kwantowym algorytmem wyszukiwania minimum stworzonym przez Dürra i Høyer [4]. Algorytm ten stanowi rozwinięcie kwantowego algorytmu wyszukiwania Grovera [5], wykorzystując uogólniony algorytm wyszukiwania opracowany przez Boyera, Brassarda, Høyer i Tappa (BBHT) [6]. W przeciwieństwie do klasycznego algorytmu Grovera, który zakłada znajomość liczby rozwiązań, algorytm ten umożliwia skuteczne wyszukiwanie również w przypadku nieznaney liczby rozwiązań.

Algorytm Dürra–Høyer realizuje wyszukiwanie minimum poprzez iteracyjne wywoływanie algorytmu BBHT w celu znalezienia elementu o wartości mniejszej od aktualnego kandydata na minimum. Po znalezieniu takiego elementu zostaje on nowym kandydatem, a proces jest powtarzany aż do momentu, gdy z dużym prawdopodobieństwem nie istnieje już element o mniejszej wartości.

Algorytm Grovera

Algorytm Grovera [5] to kwantowa procedura sprawdzania obecności klucza w bazie danych. W nim baza danych nie ma ustalonej struktury jak w klasycznych bazach gdzie dane są zorganizowane np. w struktury drzewiaste. Jego celem jest znalezienie elementu/elementów z wykorzystaniem wyroczni (*oracle*), która potrafi rozpoznać poprawne rozwiązania. W przeciwieństwie do klasycznego przeszukiwania liniowego wymagającego średnio $O(N)$ sprawdzeń, algorytm Grovera osiąga złożoność $O(\sqrt{N})$.

Ważnym ograniczeniem podstawowej wersji algorytmu jest konieczność wykonania odpowiedniej liczby iteracji. Liczba ta zależy od liczby elementów spełniających zadany warunek określony przez wyrocznię i w ogólnym przypadku nie jest znana. Zbyt mała liczba iteracji nie zapewnia wystarczającego wzmocnienia amplitudy rozwiązania, natomiast zbyt duża powoduje jej ponowne zmniejszenie, co obniża prawdopodobieństwo poprawnego wyniku. Problem ten został rozwiązany w algorytmie Boyera, Brassarda, Høyer i Tappa (BBHT), który zostanie omówiony w kolejnym podrozdziale.

ALGORYTM GROVERA

1. Przygotuj rejestr kwantowy w stanie równomiernej superpozycji wszystkich N możliwych stanów.
2. Powtarzaj operator Grovera odpowiednią liczbę razy:
 - (a) Zastosuj wyrocznie (*oracle*), która zmienia znak amplitudy stanów odpowiadających rozwiązaniom problemu.
 - (b) Zastosuj operator dyfuzji (*diffusion transform*), który odbija amplitudy wszystkich stanów względem ich średniej wartości, zwiększając amplitudę stanów oznaczonych przez wyrocznie i zmniejszając amplitudy pozostałych.
3. Wykonaj pomiar rejestru kwantowego.
4. Zwróć zmierzony stan jako wynik wyszukiwania.

Opracowanie własne na podstawie [5].

W pierwszym kroku tworzona jest superpozycja wszystkich możliwych stanów. Żeby to uzyskać w rejestrze kwantowym złożonym z n kubitów stosujemy n -krotnie bramkę Hadamarda. [7, Strona 162, Rozdział 4.4]

$$|\psi\rangle = H^{\otimes n}|00\cdots 0\rangle = (H|0\rangle)^{\otimes n} = \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right)^{\otimes n} = \frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} |i\rangle. \quad (2.3)$$

Dla stanu kubitu złożonego z $n=3$ kubitów odpowiada to temu zapisowi [7, Strona 278, Rozdział 5.6]

$$|\psi'\rangle = \frac{1}{2\sqrt{2}}(|000\rangle + |001\rangle + |010\rangle + |011\rangle + |100\rangle + |101\rangle + |110\rangle + |111\rangle). \quad (2.4)$$

Wszystkie amplitudy prawdopodobieństwa mają wartość $\frac{1}{\sqrt{N}}$ co oznacza że każdy stan jest jednakowo prawdopodobny w przypadku pomiaru.

Kolejny etap stanowi iteracyjne wykonywanie operatora Grovera. Każda iteracja składa się z dwóch operacji. Pierwszą z nich jest działanie wyroczni, która identyfikuje stany spełniające zadany warunek poprzez odwrócenie znaku ich amplitudy. Jest to funkcja która odpowiada za wskazanie szukanego elementu. Jej definicja jest następująca. [7, Strona 162, Rozdział 4.4]

$$f(i) = \begin{cases} 1, & \text{jeśli } i \text{ reprezentuje szukany element,} \\ 0, & \text{w przeciwnym przypadku.} \end{cases} \quad (2.5)$$

Przypadek w którym wyrocznia zmienia znak przy trzeciej amplitudzie można zapisać w ten sposób. [7, Strona 278, Rozdział 5.6]

$$|\psi''\rangle = \frac{1}{2\sqrt{2}}(|000\rangle + |001\rangle - |010\rangle + |011\rangle + |100\rangle + |101\rangle + |110\rangle + |111\rangle). \quad (2.6)$$

Sam układ kwantowy realizujący tą funkcję jest zależny jest już od rozwiązywanego problemu.

Drugą operacją jest operator dyfuzji, nazywany również operatorem obrotu wokół średniej. Jego zadaniem jest zwiększenie amplitud stanów oznaczonych przez wyrocznię kosztem amplitud pozostałych stanów. Powtarzanie tych dwóch operacji prowadzi do stopniowego wzrostu prawdopodobieństwa uzyskania poprawnego rozwiązania podczas pomiaru.

Po zakończeniu iteracji wykonywany jest pomiar rejestru kwantowego. Jeżeli liczba iteracji została dobrana poprawnie, otrzymany stan z wysokim prawdopodobieństwem odpowiada szukanemu rozwiązaniu.

2.4 Rodzaje grafów

Wpływ rodzaju grafu na wydajność algorytmu jest jednym z najważniejszych aspektów niniejszej pracy. Analiza zostanie przeprowadzona na czterech różnych rodzajach grafów: grafie rzadkim, średnio gęstym, gęstym oraz specjalnie skonstruowanym grafie, który dzięki odpowiednio dobranym wagom i liczbie krawędzi wymusza maksymalną liczbę relaksacji. We wszystkich analizowanych przypadkach rozpatrywane są grafy nieskierowane, bez pętli własnych, o nieujemnych wagach krawędzi. W przypadku tego ostatniego rodzaju grafu wagi krawędzi będą ściśle określone. Natomiast dla pozostałych grafów wagi będą losowane z przedziału 1 do n^2 , gdzie n oznacza liczbę wierzchołków.

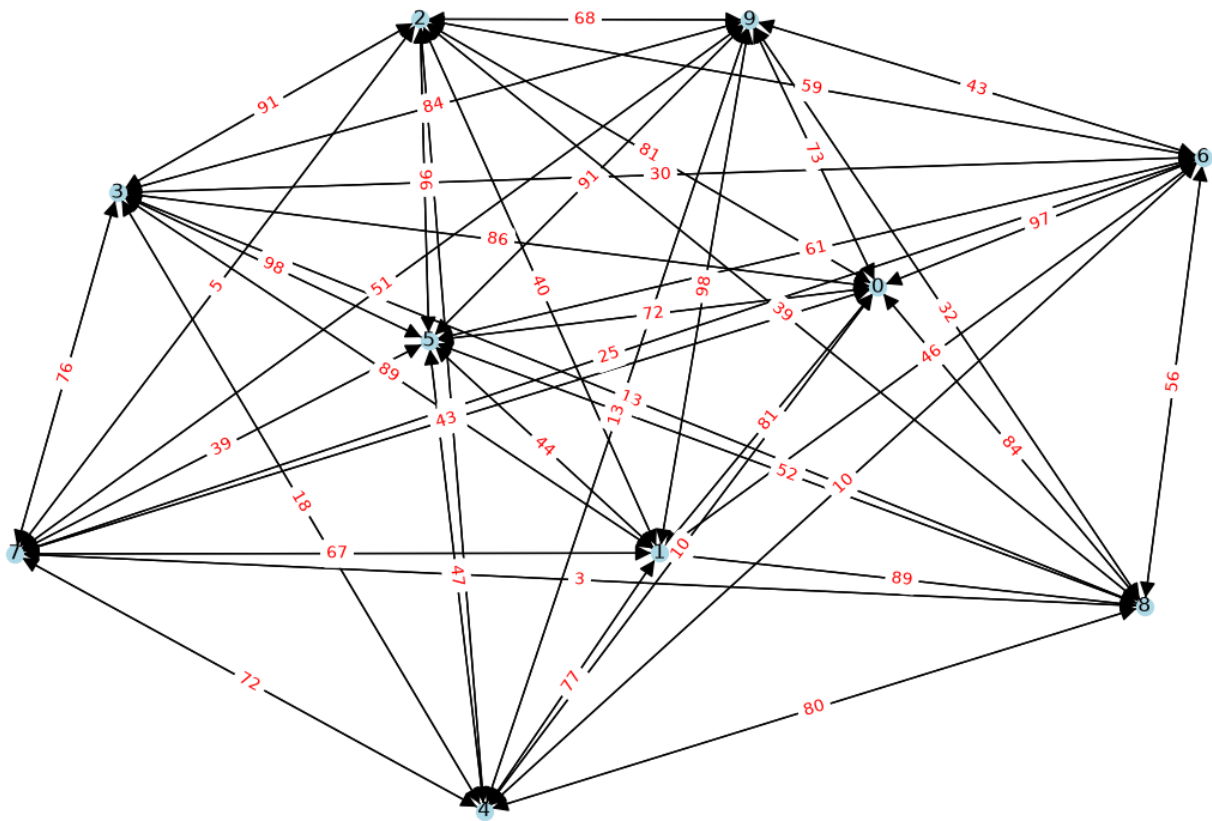
2.4.1 Graf gęsty (Dense Graph)

Graf gęsty zawiera liczbę krawędzi bliską maksymalnej.

$$|E| = O(|V|^2)[1]$$

W niniejszej pracy graf gęsty jest grafem pełnym nieskierowanym, w którym liczba krawędzi jest równa maksymalnej liczbie krawędzi dopuszczalnej dla grafu nieskierowanego bez pętli:

$$E = \frac{V(V-1)}{2}[8]$$



Przykład grafu "dense" analizowanego w pracy

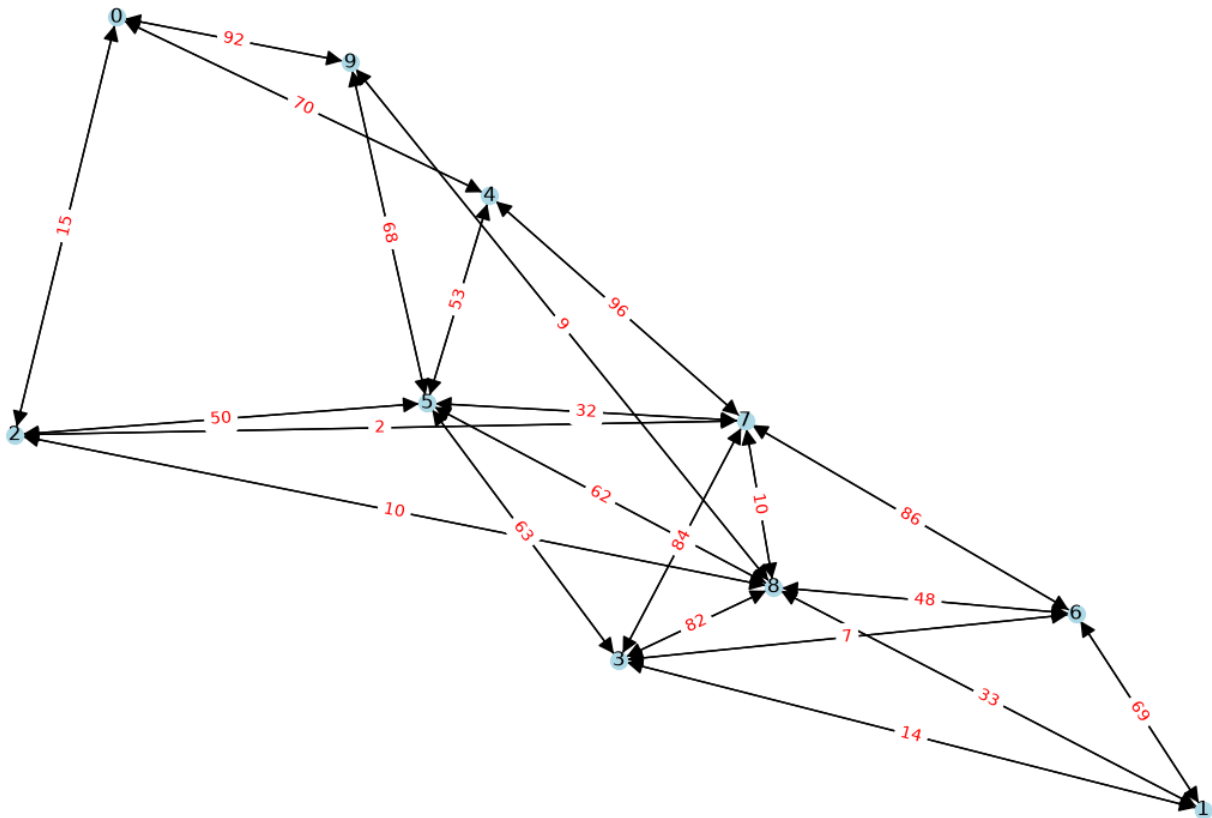
2.4.2 Graf o średniej gęstości (Half Dense)

Graf średnio gęsty zajmuje pośrednią pozycję pomiędzy grafami rzadkimi i pełnymi. Liczba krawędzi jest znacznie większa niż liczba wierzchołków, jednak nadal nie osiąga wartości maksymalnej jak w grafach gęstych.

$$|E| = O(|V|^2)[1]$$

W niniejszej pracy jako graf o średniej gęstości przyjęto graf nieskierowany, którego liczba krawędzi stanowi połowę maksymalnej liczby krawędzi:

$$E = \frac{V(V-1)}{4}$$



Przykład grafu "half-edges" analizowanego w pracy

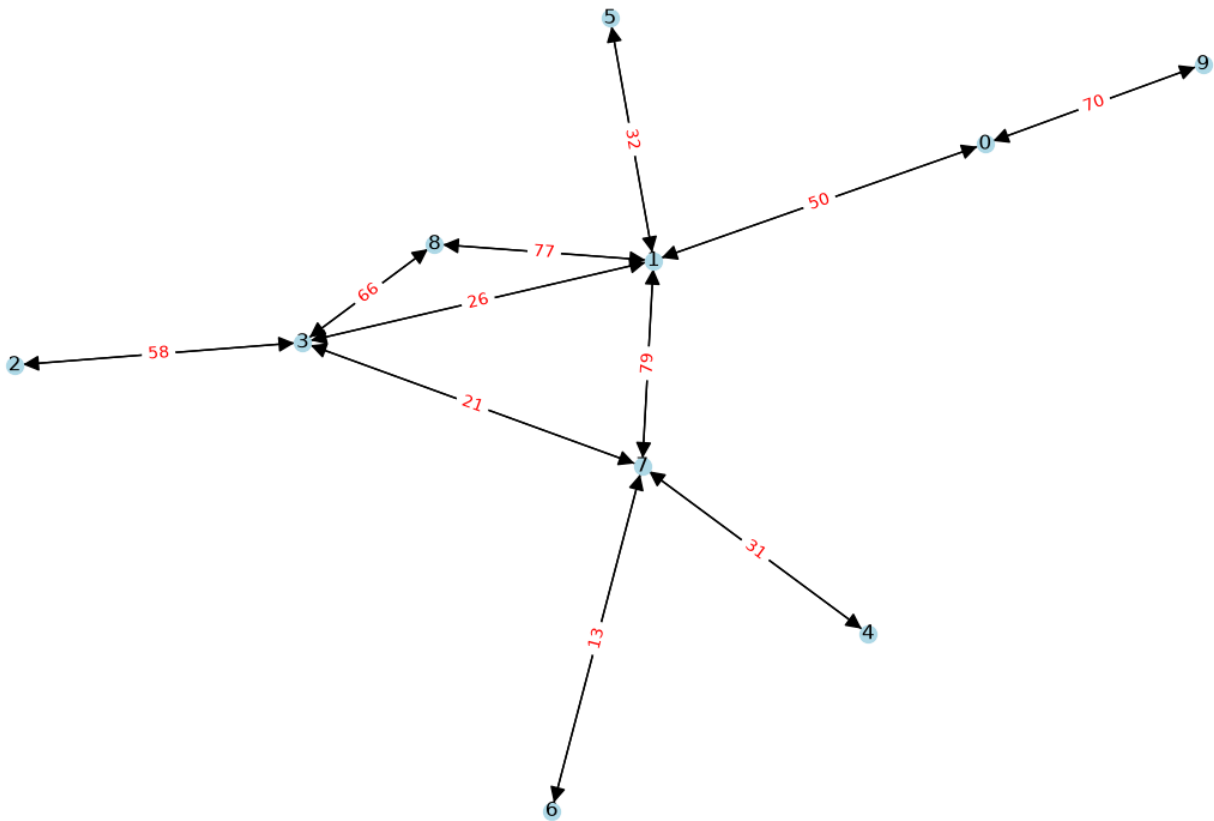
2.4.3 Graf rzadki (Sparse Graph)

Graf rzadki charakteryzuje się niewielką liczbą krawędzi względem maksymalnej liczby możliwych połączeń.

$$|E| = O(|V|)[1]$$

Przykładami grafów rzadkich są sieci drogowe, drzewa oraz wiele rzeczywistych sieci komputerowych. W niniejszej pracy jako graf rzadki przyjęto graf nieskierowany, w którym liczba krawędzi jest równa liczbie wierzchołków:

$$E = V$$



Przykład grafu "sparse" analizowanego w pracy

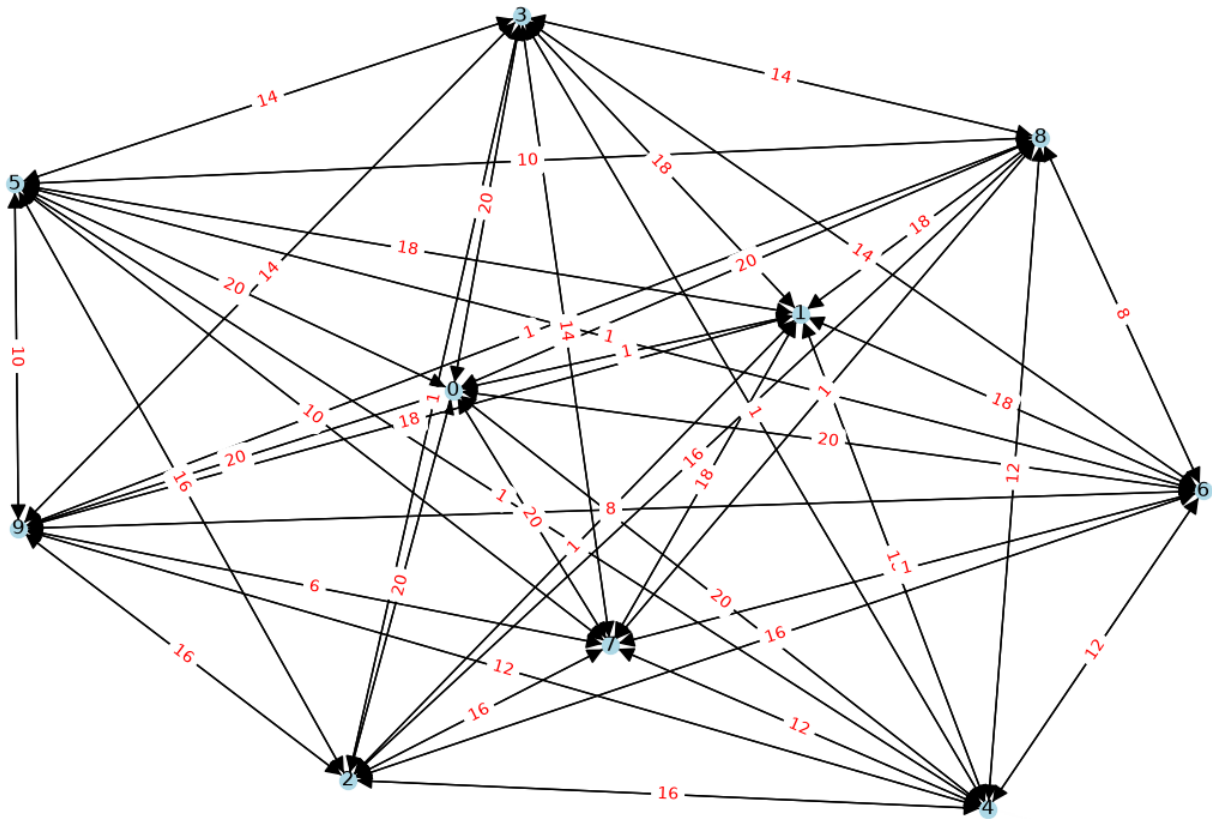
2.4.4 Przypadek szczególny (Special Case)

Przypadek szczególny jest specjalnie przygotowanym grafem pełnym. W odróżnieniu od pozostałych analizowanych grafów jego struktura oraz wagi krawędzi nie są losowe, lecz zostały dobrane w taki sposób, aby wymusić dużą liczbę operacji relaksacji wykonywanych przez algorytm Dijkstry. Tak jak graf gęsty, przypadek szczególny zawiera maksymalną liczbę krawędzi dla grafu nieskierowanego bez pętli:

$$E = \frac{V(V-1)}{2} [8]$$

Wagi krawędzi są wyznaczone zgodnie z następującą zależnością:

$$w(i, j) = \begin{cases} 1, & \text{gdy } j = i + 1, \\ 2(|V| - i), & \text{gdy } j > i + 1. \end{cases}$$



Przykład grafu "special case" analizowanego w pracy

2.5 Reprezentacja grafów

Graf może być reprezentowany na różne sposoby, przy czym do najczęściej stosowanych metod należą macierz sąsiedztwa oraz lista sąsiedztwa.

Macierz sąsiedztwa przedstawia graf w postaci dwuwymiarowej tablicy, w której element znajdujący się na przecięciu i -tego wiersza i j -tej kolumny określa istnienie krawędzi pomiędzy odpowiadającymi im wierzchołkami lub przechowuje jej wagę. Reprezentacja ta zawiera informacje o wszystkich możliwych parach wierzchołków, niezależnie czy są one połączone krawędzią.

Lista sąsiedztwa przypisuje każdemu wierzchołkowi listę jego bezpośrednich sąsiadów. W przypadku grafu ważonego każdy element tej listy składa się z numeru sąsiedniego wierzchołka oraz wagi krawędzi łączącej oba wierzchołki.

W implementacji przedstawionej w niniejszej pracy graf został zapisany w postaci listy sąsiedztwa. Dla każdego wierzchołka przechowywana jest lista wszystkich wierzchołków z nim połączonych wraz z odpowiadającymi im wagami krawędzi. W przypadku grafu nieskierowanego każda krawędź występuje dwukrotnie – na liście sąsiedztwa obu wierzchołków, które łączy.

Algorithm 1 Algorytm Dijkstry

Require: Graf $G = (V, E)$, funkcja wag $w : E \rightarrow \mathbb{R}_{\geq 0}$, wierzchołek źródłowy s

Ensure: Najkrótsze odległości $d[v]$ od s do każdego $v \in V$ oraz poprzednicy $\pi[v]$

```

1: for all  $v \in V$  do
2:    $d[v] \leftarrow \infty$ 
3:    $\pi[v] \leftarrow \text{NIL}$ 
4: end for
5:  $d[s] \leftarrow 0$ 
6:  $Q \leftarrow V$ 
7: while  $Q \neq \emptyset$  do
8:    $u \leftarrow \text{EXTRACTMIN}(Q, d)$ 
9:   for all  $v \in \text{SĄSIEDZI}(u)$  do
10:    if  $v \in Q$  and  $d[v] > d[u] + w(u, v)$  then
11:       $d[v] \leftarrow d[u] + w(u, v)$ 
12:       $\pi[v] \leftarrow u$ 
13:    end if
14:  end for
15: end while
16: return  $(d, \pi)$ 
```

$$f(i) = \begin{cases} 1, & \text{jeśli } i \text{ reprezentuje szukany element,} \\ 0, & \text{w przeciwnym przypadku.} \end{cases}$$

02

$$U_f(|i\rangle|j\rangle) = |i\rangle|j \oplus f(i)\rangle.$$

1

$$1 \oplus f(i) = \begin{cases} 0, & \text{jeśli } i \text{ reprezentuje szukany element,} \\ 1, & \text{w przeciwnym przypadku.} \end{cases}$$

2

$$U_f(|i\rangle|-\rangle) = \frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} (-1)^{f(i)} |i\rangle|-\rangle.$$

3

$$A = 2|\psi\rangle\langle\psi| - I.$$

4

$$|\psi\rangle = H^{\otimes n} |0\rangle^{\otimes n} = \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}} \right)^{\otimes n} = \frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} |i\rangle, \quad N = 2^n. \quad (2.7)$$

5

$$f(i) = \begin{cases} 1, & i = i_0, \\ 0, & i \neq i_0. \end{cases} \quad (2.8)$$

6

$$U_f(|i\rangle|j\rangle) = |i\rangle|j \oplus f(i)\rangle. \quad (2.9)$$

7

$$|-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}$$

8

$$U_f(|i\rangle|-\rangle) = (-1)^{f(i)} |i\rangle|-\rangle. \quad (2.10)$$

9

$$U_f \left(\frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} |i\rangle|-\rangle \right) = \frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} (-1)^{f(i)} |i\rangle|-\rangle. \quad (2.11)$$

10

$$A = 2|\psi\rangle\langle\psi| - I. \quad (2.12)$$

11

$$|i_f\rangle$$

13

14

15

16

17

18

20

21

[illegible]

Bibliografia

- [1] B. Bollobas, O. Riordan. Metrics for sparse graphs. Surveys in Combinatorics 2009, LMS Lecture Notes Series 365, CUP 2009, pp. 211–287, 2008. <https://arxiv.org/abs/0708.1919>.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009. <https://www.cs.mcgill.ca/~akroitt/math/compsci/Cormen%20Introduction%20to%20Algorithms.pdf>.
- [3] Dijkstra, E.W. A note on two problems in connexion with graphs. Numer. Math. 1, 269–271, 1959. <https://ir.cwi.nl/pub/9256/9256D.pdf>.
- [4] Christoph Dürr and Peter Høyer. A quantum algorithm for finding the minimum. <https://arxiv.org/abs/quant-ph/9607014>, 1996.
- [5] Lov K. Grover. A fast quantum mechanical algorithm for database search. Proceedings of the 28th Annual ACM Symposium on Theory of Computing, 1996. <https://arxiv.org/abs/quant-ph/9605043>.
- [6] P. Høyer M. Boyer, G. Brassard and A. Tapp. Tight bounds on quantum searching. Fortschritte Der Physik, 1996. <https://arxiv.org/abs/quant-ph/9605034>.
- [7] Sawerwain Marek Wiśniewska Joanna. *Informatyka kwantowa. Wybrane obwody i algorytmy*. Wydawnictwo Naukowe PWN, 1 edition, 2015.
- [8] Wolfram MathWorld. Complete graph. <https://mathworld.wolfram.com/CompleteGraph.html>. Accessed: 04.08.2026.